

React refresher for backend developers

A guided tour of modern React, one concept per lesson

- [About this guide](#)
- [Lesson: Home \(useEffect baseline\)](#)
- [Lesson: Items \(loader + discriminated union\)](#)
- [Lesson: TanStack Query](#)
- [Lesson: React 19 form hooks](#)
- [Lesson: client state \(Context + Zustand\)](#)
- [Lesson: mutations + cache invalidation](#)
- [Lesson: composition + useReducer](#)
- [Lesson: session management](#)

About this guide

A small full-stack app (React + Vite + React Router on the front, Express on Bun for the API) where each route teaches one modern React concept, with explicit parallels to how it'd work in production.

Each lesson below has two parts: a short write-up (what it teaches, the production parallel, what to try, when not to use it), followed by the annotated source code. Comments use a fixed vocabulary:

- **CONCEPT:** the React idea the block teaches
- **PROD:** the parallel to a real production system
- **GOTCHA:** the trap people hit

Lesson: Home (useEffect baseline)

Route: /

client/src/routes/Home.tsx

```
// Hooks are React's built-in functions for "remembering" data (useState)
// and running side-effects like network calls (useEffect).
import { useEffect, useState } from "react";
import { delay } from "../lib/delay";

// TypeScript type describing the shape of one item from the API.
type Item = { id: number; name: string };

// A component is just a function that returns the UI (JSX) to render.
// React re-runs this entire function whenever its state changes.
export default function Home() {
  // useState = component memory that survives re-renders.
  // Returns [currentValue, setterFn]. Update ONLY via the setter -
  // calling setItems(...) is what tells React to re-render. Never assign
  // `items = ...` directly; that won't update the screen.
  const [items, setItems] = useState<Item[]>([]);
  const [error, setError] = useState<string | null>(null);
  // With useEffect fetching, YOU track loading yourself. Starts true, flips
  // false when the fetch settles. This is the manual work a loader removes.
  const [loading, setLoading] = useState(true);

  // useEffect runs AFTER the component renders - the right place for
  // side-effects like fetching data. The empty [] dependency array means
  // "run this once, when the component first mounts." (No array at all
  // would re-run it every render → fetch → setState → render → infinite loop.)
  //
  // The effect callback itself canNOT be async: an async function returns a
  // Promise, but React expects the callback to return either nothing or a
  // cleanup function. So we define an async function inside and call it.
  useEffect(() => {
    async function load() {
      try {
        await delay(1000); // artificial delay so the loading state is visible
        const res = await fetch("/api/items");
        // fetch only rejects on network failure, not on HTTP errors,
        // so check res.ok and throw manually for 4xx/5xx responses.
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        // Store the data in state - this triggers a re-render with the items.
        setItems(await res.json());
      } catch (err) {
        // Any thrown error (network or HTTP) lands here and goes into state.
        setError(String(err));
      } finally {
        // Runs whether we succeeded or failed - turn the loading flag off.
        setLoading(false);
      }
    }
    // Kick it off. The effect callback returns undefined (not the Promise),
    // which keeps React's cleanup contract happy.
    load();
  }, []);

  // The return value is JSX: HTML-like syntax that compiles to JS.
  // It re-runs and updates the DOM automatically whenever state changes.
  return (
```

```

<section>
  <h1>Home</h1>
  <p>Items fetched from the API:</p>
  { /* {expr} embeds JS in JSX. Render the error line only if error is set. */
  {error && <p className="error">Failed to load: {error}</p>}
  { /* Show a placeholder while loading is true - the component-managed
      loading state you have to wire up yourself with useEffect. */
  {loading && <p>Loading...</p>}
  <ul>
    { /* .map() turns each data item into an <li>. `key` must be a stable,
        unique id so React can track list items efficiently across renders. */
    {items.map((item) => (
      <li key={item.id}>{item.name}</li>
    ))}
  </ul>
</section>
);
}

```

Lesson: Items (loader + discriminated union)

Route: `/items`

client/src/routes/Items.tsx

```
import { useLoaderData } from "react-router-dom";
import { delay } from "../lib/delay";

// Shape of one item from the API.
type Item = { id: number; name: string };

// A DISCRIMINATED UNION: the loader returns ONE of these two shapes. The
// literal `ok: true` / `ok: false` field is the "discriminant" – the tag
// TypeScript uses to tell the variants apart. Checking `data.ok` lets the
// compiler narrow to exactly one variant, so `items` and `error` are each
// only reachable in their own branch. No casts, no `as` – the type system
// does the work.
type LoaderResult =
  | { ok: true; items: Item[] }
  | { ok: false; error: string };

// A loader is a plain async function tied to the route in the routing table.
// React Router runs it BEFORE rendering this component, waits for it to
// resolve, then renders – so the data is already here on first paint.
//
// The explicit `Promise<LoaderResult>` return type is what makes the union
// flow through to the component: every return below must match one variant,
// and useLoaderData<typeof itemsLoader>() infers this same type – no cast.
//
// NOTE: an alternative React Router idiom is to THROW on failure; the router
// then renders the route's `errorElement` (read via useRouteError()) and this
// component only ever sees success. Returning the union instead keeps both
// cases in one place, which is the clearer demonstration of narrowing.
export async function itemsLoader(): Promise<LoaderResult> {
  await delay(1000); // artificial delay – navigation blocks here until done
  try {
    const res = await fetch("/api/items");
    if (!res.ok) return { ok: false, error: `HTTP ${res.status}` };
    // Pure-TS version: we trust the JSON is Item[]. (A runtime schema check,
    // e.g. Zod, would belong right here to make the success branch airtight.)
    const items: Item[] = await res.json();
    return { ok: true, items };
  } catch (err) {
    return { ok: false, error: String(err) };
  }
}

export default function Items() {
  // Typed inference from the loader's signature – no `as` cast. The generic
  // <typeof itemsLoader> tells useLoaderData the return type, so `data` is
  // LoaderResult here purely through inference.
  const data = useLoaderData<typeof itemsLoader>();

  // The discriminant check. After this `if`, TypeScript KNOWS `data` is the
  // error variant inside the block, and the success variant after it – try
  // accessing data.items here and it's a compile error.
  if (!data.ok) {
    return (
      <section>
        <h1>Items (loader)</h1>
      </section>
    );
  }
}
```

```

    <p className="error">Failed to load: {data.error}</p>
  </section>
);
}

// Past the guard, `data` is narrowed to { ok: true; items: Item[] } - so
// data.items is fully typed and safe to use with no cast.
return (
  <section>
    <h1>Items (loader)</h1>
    <p>
      Same data as Home, but fetched by a route <code>loader</code> before
      render - notice there's no loading flash.
    </p>
    <ul>
      {data.items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  </section>
);
}

```

Lesson: TanStack Query

Route: `/query` · **Concept:** server-state management

What it teaches

`useQuery` replaces the entire `useEffect` + 3× `useState` dance from the `/` route with one declarative call. You describe *what* data you want (`queryKey`) and *how* to get it (`queryFn`); the library owns caching, loading/error state, deduplication, and background revalidation.

Compare the two routes

	<code>/</code> (<code>useEffect</code>)	<code>/query</code> (TanStack Query)
State you manage	data + loading + error by hand	none - the hook returns it
Caching	none (refetches every mount)	cached by <code>queryKey</code>
Duplicate requests	each component fetches again	deduped automatically
Background refresh	you'd build it	built in (<code>staleTime</code>)

The production parallel

- `queryKey` = **cache namespace**. Same key → shared cache + shared in-flight request across the whole app. This is the mechanism mutations use to invalidate.

- `staleTime` / `retry` / **refetch-on-focus** are policy you set once on the `QueryClient` - the client-side analogue of HTTP cache headers + retry/backoff.
- `queryFn` **is your API client layer** - pure async functions, no React, reusable.
- This is why teams stop hand-rolling `useEffect` fetches: at scale you need caching, dedup, and invalidation, and reinventing them per-component is a bug farm.

Try this

1. Visit `/query`, then click **Refetch** - watch “Revalidating...” while old data stays.
2. Navigate to `/about` and back **within 30s** - data is instant (served from cache, `isPending` never true).
3. Wait >30s (past `staleTime`), come back - instant data PLUS a background refetch.
4. Break the URL in `fetchItems` (e.g. `/api/itemzzz`) - see the error state + retry.

When NOT to use it

- Pure **client** state (UI toggles, form drafts, selected tab) - that’s not server state; use `useState` / `Context` / `Zustand` (`/dashboard` lesson).
- Trivial one-shot fetches where you’ll never cache, refetch, or share. A loader or `useEffect` is fine. `TanStack` earns its keep when data is reused and must stay fresh.

Source

client/src/routes/query/Query.tsx

```
import { useQuery } from "@tanstack/react-query";
import { delay } from "../../lib/delay";

type Item = { id: number; name: string };

// CONCEPT: A plain async function that fetches and returns typed data. Notice
// it has NO React in it – no hooks, no state. It's just "how to get the data."
// TanStack Query calls this for you and manages everything around it.
// PROD: this is your API client layer. In a real app these live in one place
// (e.g. src/api/) and are reused by every query, so caching keys line up.
async function fetchItems(): Promise<Item[]> {
  await delay(1000); // artificial delay so loading/cached states are visible
  const res = await fetch("/api/items");
  // GOTCHA: TanStack treats a REJECTED promise as the error state. fetch does
  // NOT reject on 4xx/5xx, so you must throw yourself – otherwise a 500 looks
  // like success with garbage data.
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

export default function Query() {
  // CONCEPT: useQuery replaces ALL of Home's manual machinery – the useState
  // for data, the useState for loading, the useState for error, AND the
  // useEffect – with one call. You give it:
  // - queryKey: the cache identity for this data (like a cache key / URL)
  // - queryFn: how to fetch it
  // and it hands back data + status flags, fully typed from fetchItems.
  //
  // PROD: queryKey is the heart of the system. Two components using the same
  // key share one cache entry and one in-flight request (automatic dedup).
  // Mutations later invalidate BY key to force a refetch. Keys are usually
  // structured like ["items"] or ["items", { filter }] – think cache namespacing.
  const { data, isPending, isError, error, isFetching, refetch } = useQuery({
    queryKey: ["items"],
    queryFn: fetchItems,
  });

  // CONCEPT: isPending = no cached data yet (first load). isFetching = a request
  // is in flight, INCLUDING background refetches when you already have data.
  // GOTCHA: the distinction matters – on a background refetch you keep showing
  // the old data (isFetching true, isPending false) instead of a blank loader.
  if (isPending) return <Shell><p>Loading...</p></Shell>;
  if (isError) return <Shell><p className="error">Failed: {String(error)}</p></Shell>;

  return (
    <Shell>
      {/* isFetching here = a background revalidation while showing cached data */}
      {isFetching && <p className="loading-bar">Revalidating...</p>}
      <ul>
        {data.map((item) => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    </Shell>
  );
}
```

```

    </ul>
    { /* CONCEPT: refetch() forces a fresh fetch on demand. Click it, then
       navigate away and back within 30s (staleTime) - you'll see the cached
       data appear INSTANTLY with no loader, the payoff of the cache. */
      <button onClick={() => refetch()}>Refetch</button>
    </Shell>
  );
}

// Small layout wrapper so the three return branches share the same heading.
function Shell({ children }: { children: React.ReactNode }) {
  return (
    <section>
      <h1>Query (TanStack Query)</h1>
      <p>
        Same data as Home, but fetched with <code>useQuery</code>. Try: navigate
        away and back within 30s - cached, instant, no loader.
      </p>
      {children}
    </section>
  );
}

```

client/src/lib/queryClient.ts

```

import { QueryClient } from "@tanstack/react-query";

// CONCEPT: The QueryClient is TanStack Query's cache + config. One instance for
// the whole app; every useQuery/useMutation reads and writes through it.
//
// PROD: This is where teams set global policy - how long data stays "fresh"
// before a background refetch, retry/backoff on failure, refetch-on-focus, etc.
// Think of it as the client-side equivalent of your HTTP cache / connection-pool
// config: set once, applied everywhere.
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      // CONCEPT: "fresh" data is served from cache with NO network call. After
      // staleTime it becomes "stale" and the next use triggers a background
      // refetch (you still see cached data instantly while it revalidates).
      // PROD: tune per data volatility - a user profile might be minutes,
      // a stock ticker seconds. Here 30s so you can observe the behavior.
      staleTime: 30_000,
      // PROD: real APIs blip. TanStack retries failed queries with backoff by
      // default (3x). Lowered to 1 here to keep the demo's errors quick to see.
      retry: 1,
    },
  },
});

```

Lesson: React 19 form hooks

Route: /new-item · **Concept:** native form lifecycle + optimistic UI **Backend:** POST /api/items (Express)

What it teaches

Three React 19 hooks that replace hand-rolled form plumbing:

- `useActionState(action, initial)` - owns the submit lifecycle. You write an async `(prevState, formData) => newState`; React tracks pending and re-runs it on `<form action={...}> submit`. Replaces `onSubmit + useState(loading) + try/catch`.
- `useOptimistic(real, reducer)` - render a *predicted* result while the action is in flight; React auto-reverts to the real state when it settles (or fails).
- `useFormStatus()` - a child component reads the parent form's `pending` with no prop drilling (e.g. the submit button disables itself).

The full-stack round trip

1. You submit → optimistic row appears instantly (`id: -1`, “saving…”).
2. `POST /api/items` runs (1s delay) → server validates, returns the real record.
3. Success → action commits the canonical server item (real `id`).
4. Failure (name contains `fail` → 422) → action returns the unchanged list → React drops the optimistic row automatically (**rollback**).

The production parallel

- **Optimistic UI** is what makes apps feel instant (think: sending a chat message, liking a post). The pattern is always: show the guess, reconcile with the server, roll back on failure. `useOptimistic` is the native version; TanStack Query's `useMutation` `onMutate` / `onError` / `onSettled` is the same idea with cache integration.
- **Trust the server's record, not your guess.** The optimistic row uses a fake `id: -1`; the committed row uses the server's real `id`. Never let the placeholder become the source of truth.
- **Validate on BOTH sides.** Client guard for UX (empty name), server guard for correctness (400/422). The server is the only one you can trust - the client check is just to save a round trip.
- **POST returns 201 + the created resource** - REST convention so the client gets the authoritative record (id, server-set fields) without a second GET.

Try this

1. Add a normal item → watch “saving…” for 1s, then it commits.
2. Add an item named `fail me` → optimistic row appears, then **vanishes** when the server returns 422 (rollback). An `alert` shows the error (a real app = inline UI).
3. Submit empty → client guard blocks it before any request.

When NOT to use it

- **Complex forms** (many fields, cross-field rules, dynamic arrays) - use React Hook Form + Zod; it's uncontrolled (fewer re-renders) and has richer validation.
- When the new data must update a **shared cache** other screens read - combine with TanStack Query's `useMutation` + `invalidateQueries(["items"])` so `/query` updates too.

Source

client/src/routes/new-item/NewItem.tsx

```
import { useActionState, useOptimistic, useRef } from "react";
import { useFormStatus } from "react-dom";

type Item = { id: number; name: string };

// CONCEPT: the API call lives outside the component – a plain async function,
// same discipline as the queryFn in the /query lesson.
// PROD: returns the canonical server record (with the real id). The client
// should trust THIS, not the optimistic guess it showed a moment ago.
async function createItem(name: string): Promise<Item> {
  const res = await fetch("/api/items", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name }),
  });
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error ?? `HTTP ${res.status}`);
  }
  return res.json();
}

export default function NewItem() {
  const formRef = useRef<HTMLFormElement>(null);

  // CONCEPT: useActionState manages the whole submit lifecycle. You give it an
  // async "action" (state, formData) => newState. React tracks pending for you
  // and re-runs the action when the <form action={...}> is submitted.
  // The first slot is the latest returned state; the third is the bound action.
  // PROD: this is the React 19 native replacement for wiring up onSubmit +
  // useState(loading) + try/catch by hand. For COMPLEX forms (many fields,
  // cross-field validation) you'd still reach for React Hook Form + Zod.
  const [items, submitAction] = useActionState<Item[], FormData>(
    async (prev, formData) => {
      const name = String(formData.get("name") ?? "").trim();
      if (!name) return prev; // trivial client-side guard
      try {
        const created = await createItem(name);
        formRef.current?.reset();
        return [...prev, created]; // commit the REAL server record
      } catch (err) {
        // GOTCHA: returning prev here discards the optimistic row (rollback).
        // We surface the error via the optimistic layer below instead of state.
        alert(String(err)); // demo-simple; a real app shows inline error UI
        return prev;
      }
    },
    [],
  );

  // CONCEPT: useOptimistic shows a temporary, predicted result WHILE the action
  // is in flight, then automatically reverts to the real `items` when it settles.
```

```

// optimisticItems = the list to render; addOptimistic queues a fake entry.
// PROD: this is "optimistic UI" - the row appears instantly (id: -1 placeholder)
// for a snappy feel; if the server rejects, React drops it on its own when the
// action returns the unchanged list. Same idea as TanStack's onMutate/rollback.
const [optimisticItems, addOptimistic] = useOptimistic(
  items,
  (current, newName: string) => [...current, { id: -1, name: newName }],
);

return (
  <section>
    <h1>New item (React 19 form hooks)</h1>
    <p>
      Add an item. It appears instantly (optimistic) before the 1s server
      round-trip. Type a name containing <code>fail</code> to watch the server
      reject it and the optimistic row roll back.
    </p>

    {/ * CONCEPT: React 19 forms use action={fn} instead of onSubmit. The form
      auto-resets pending state and passes FormData to the action. */}
    <form
      ref={formRef}
      action={({formData}) => {
        // Fire the optimistic update first, then the real action.
        addOptimistic(String(formData.get("name")) ?? "");
        submitAction(formData);
      }}
    >
      <input name="name" placeholder="Item name" autoComplete="off" />
      <SubmitButton />
    </form>

    <ul>
      {optimisticItems.map((item, i) => (
        // id is -1 for the in-flight optimistic row; fall back to index key.
        <li key={item.id === -1 ? `pending-${i}` : item.id}>
          {item.name}
          {item.id === -1 && <em> (saving...)</em>}
        </li>
      ))}
    </ul>
  </section>
);
}

// CONCEPT: useFormStatus reads the pending state of the NEAREST parent <form>,
// from a child component - no prop drilling. Must be in a child, not the form
// component itself.
// PROD: lets shared UI (a submit button, a spinner) know it's submitting without
// the parent passing a `loading` prop down. Great for design-system buttons.
function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending}>
      {pending ? "Adding..." : "Add"}
    </button>
  );
}

```

```
);  
}
```

server/src/index.ts

```
import express from "express";
import { createHmac, randomBytes } from "node:crypto";

const app = express();
const PORT = Number(process.env.PORT) || 3001;

app.use(express.json());

// =====
// SESSION / AUTH LESSON (see client/src/routes/auth/LESSON.md)
//
// PROD: in a real app you would NOT hand-write any of this – you'd use a
// library. Reach for Better Auth (https://better-auth.com) by default, or a
// hosted service (Clerk, Auth0) / Supabase Auth depending on the setup. This
// minimal version exists to show what those libraries do internally: issue a
// short-lived access token + a long-lived, revocable refresh token, both in
// httpOnly cookies, and refresh the access token when it expires.
// =====

// PROD: signing secret comes from env/secrets manager, never hard-coded.
const SECRET = "dev-only-secret-do-not-use-in-prod";
const ACCESS_TTL_MS = 10_000; // 10s so you can SEE expiry+refresh in the demo
const DEMO_USER = { id: 1, email: "demo@example.com", password: "password" };

// PROD: this Set is the "stateful" half – the refresh-token store (Redis/DB in
// prod). Storing refresh tokens is what lets us REVOKE them (logout everywhere),
// the revocation power a pure stateless JWT can't give you.
const validRefreshTokens = new Set<string>();

// A tiny signed token: base64(payload).hmac. Stands in for a real JWT.
// PROD: use a vetted lib (jsonwebtoken / jose) – it handles alg, exp, claims.
function sign(payload: object): string {
  const body = Buffer.from(JSON.stringify(payload)).toString("base64url");
  const mac = createHmac("sha256", SECRET).update(body).digest("base64url");
  return `${body}.${mac}`;
}

function verify(token: string | undefined): any | null {
  if (!token) return null;
  const [body, mac] = token.split(".");
  if (!body || !mac) return null;
  const expected = createHmac("sha256", SECRET).update(body).digest("base64url");
  if (mac !== expected) return null; // tampered/invalid signature
  const payload = JSON.parse(Buffer.from(body, "base64url").toString());
  if (payload.exp && Date.now() > payload.exp) return null; // expired
  return payload;
}

// Minimal cookie parser (PROD: use cookie-parser / your framework's built-in).
function getCookie(req: express.Request, name: string): string | undefined {
  const raw = req.headers.cookie;
  if (!raw) return undefined;
  for (const part of raw.split(";")) {
    const [k, ...v] = part.trim().split("=");
    if (k === name) return decodeURIComponent(v.join("="));
  }
}
```

```

    return undefined;
}

// Set an httpOnly cookie. The httpOnly flag is the whole point: JS on the page
// CANNOT read it, so an XSS injection can't steal the token.
// PROD: also set Secure (HTTPS only) and a sensible SameSite for CSRF defense.
function setAuthCookie(res: express.Response, name: string, value: string) {
    res.cookie(name, value, {
        httpOnly: true,
        sameSite: "lax", // CSRF mitigation; pairs with a CSRF token in real apps
        // secure: true, // enable behind HTTPS in production
    });
}

// POST /api/login - exchange credentials for access + refresh cookies.
app.post("/api/login", (req, res) => {
    const { email, password } = req.body ?? {};
    if (email !== DEMO_USER.email || password !== DEMO_USER.password) {
        return res.status(401).json({ error: "invalid credentials" });
    }
    const accessToken = sign({ sub: DEMO_USER.id, exp: Date.now() + ACCESS_TTL_MS });
    const refreshToken = randomBytes(24).toString("hex");
    validRefreshTokens.add(refreshToken); // remember it so we can revoke later
    setAuthCookie(res, "accessToken", accessToken);
    setAuthCookie(res, "refreshToken", refreshToken);
    res.json({ user: { id: DEMO_USER.id, email: DEMO_USER.email } });
});

// POST /api/refresh - mint a new access token if the refresh token is valid.
// PROD: rotate the refresh token here too (issue a new one, invalidate the old)
// to limit the damage of a stolen refresh token.
app.post("/api/refresh", (req, res) => {
    const refreshToken = getCookie(req, "refreshToken");
    if (!refreshToken || !validRefreshTokens.has(refreshToken)) {
        return res.status(401).json({ error: "invalid refresh token" });
    }
    const accessToken = sign({ sub: DEMO_USER.id, exp: Date.now() + ACCESS_TTL_MS });
    setAuthCookie(res, "accessToken", accessToken);
    res.json({ ok: true });
});

// POST /api/logout - clear cookies AND revoke the refresh token server-side.
app.post("/api/logout", (req, res) => {
    const refreshToken = getCookie(req, "refreshToken");
    if (refreshToken) validRefreshTokens.delete(refreshToken); // revocation
    res.clearCookie("accessToken");
    res.clearCookie("refreshToken");
    res.json({ ok: true });
});

// GET /api/me - the protected endpoint. 401 if no valid access token; the
// client treats that 401 as "try refreshing, then retry".
app.get("/api/me", (req, res) => {
    const payload = verify(getCookie(req, "accessToken"));
    if (!payload) return res.status(401).json({ error: "not authenticated" });
    res.json({ user: { id: DEMO_USER.id, email: DEMO_USER.email } });
});

```

```

// In-memory store so POSTed items actually persist for the session.
// PROD: this is your database. Swap for Postgres/etc; the route shape is the same.
type Item = { id: number; name: string };
let items: Item[] = [
  { id: 1, name: "First item" },
  { id: 2, name: "Second item" },
  { id: 3, name: "Third item" },
];
let nextId = 4;

// Small helper to simulate real network/DB latency so the client's optimistic
// UI is observable (the row appears instantly, before this resolves).
const delay = (ms: number) => new Promise((r) => setTimeout(r, ms));

// GET /api/health - liveness check
app.get("/api/health", (_req, res) => {
  res.json({ status: "ok", uptime: process.uptime() });
});

// GET /api/items - current list
app.get("/api/items", (_req, res) => {
  res.json(items);
});

// POST /api/items - create one item
app.post("/api/items", async (req, res) => {
  await delay(1000); // simulate latency so optimistic UI is visible

  const name = (req.body?.name ?? "").trim();

  // PROD: validate at the boundary and return 400 on bad input. Never trust the
  // client. (This is the server-side half of the client's Zod/Either instinct.)
  if (!name) {
    return res.status(400).json({ error: "name is required" });
  }
  // Contrived rule so you can watch an optimistic update get ROLLED BACK:
  // any name containing "fail" is rejected by the server.
  if (name.toLowerCase().includes("fail")) {
    return res.status(422).json({ error: `rejected: "${name}"` });
  }

  const item: Item = { id: nextId++, name };
  items.push(item);
  res.status(201).json(item); // 201 Created + the canonical server record
});

app.listen(PORT, () => {
  console.log(`API listening on http://localhost:${PORT}`);
});

```

Lesson: client state (Context + Zustand)

Route: /dashboard · **Concept:** client state, and how it differs from server state

The headline distinction (the #1 architecture decision)

	Server state	Client state
Lives where	on the server, you cache a copy	only in the browser
Examples	items list, user profile, orders	theme, modal open, wizard step, cart
Tool	TanStack Query (/query)	<code>useState</code> / Context / Zustand
Key concern	staleness, refetch, invalidation	sharing without prop-drilling, re-renders

Mixing these up is the most common architecture mistake. Don't put server data in Zustand/Redux (you'll reinvent caching badly); don't put UI toggles in TanStack Query.

Two tools, shown side by side

React Context (theme toggle) - React's built-in, zero deps. - `createContext` → `<Provider value={...}>` → `useContext`. - Best for app-wide, rarely-changing values: **auth/user, theme, locale, feature flags**. - GOTCHA: every consumer re-renders on ANY change - no selective subscription.

Zustand (counter) - the pragmatic 2026 default for client state. - `create()` a store OUTSIDE React; no Provider needed. - Components subscribe with a **selector** (`useStore(s => s.count)`) → re-render only when their slice changes. This is the scaling advantage over Context.

The production parallel

- **Context = configuration/identity** that's read widely but written rarely. Auth provider and theme provider near the app root is the textbook pattern.
- **Zustand = interactive global UI state** touched by many unrelated components (cart, sidebar, multi-step flow). Selectors keep re-renders surgical.
- **Skip Redux Toolkit** unless you're on a large team that needs enforced patterns, middleware, and time-travel debugging - Zustand covers the same concept with far less ceremony. Learn whichever the team actually uses.
- With TanStack Query owning server state, the *amount* of client state you need shrinks a lot - many apps end up with just a small Zustand store + a couple Contexts.

Try this

1. Toggle the theme - the section recolors (Context value flows to the consumer).
2. Click +1 a few times, navigate to `/about` and back - count **resets** (client state is in-memory; it dies on unmount/reload, unlike server state).

3. Note `CounterButtons` reads only the actions: it never re-renders when the count changes. Open React DevTools' "Highlight updates" to see only `CounterDisplay` flash.

When to use which

- One component's state → `useState` .
- A handful of components, rarely-changing → **Context**.
- Many components, frequently-changing, or you want selectors → **Zustand**.
- It's data from your API → none of these - **TanStack Query** (/query).

Source

client/src/routes/dashboard/Dashboard.tsx

```
import { ThemeProvider, useTheme } from "../ThemeContext";
import { useCounterStore } from "../counterStore";

export default function Dashboard() {
  // CONCEPT: the Provider wraps only the subtree that needs the theme. You could
  // hoist it to the app root (like in main.tsx) if the whole app needed it.
  return (
    <ThemeProvider>
      <DashboardInner />
    </ThemeProvider>
  );
}

function DashboardInner() {
  // Context: read via the typed hook. This component re-renders on theme change.
  const { theme, toggle } = useTheme();

  return (
    <section
      style={{
        background: theme === "dark" ? "#1a1a1a" : "#fff",
        color: theme === "dark" ? "#f5f5f5" : "#1a1a1a",
        padding: "1rem",
        borderRadius: 6,
        transition: "background 0.2s, color 0.2s",
      }}
    >
      <h1>Dashboard (client state)</h1>
      <p>
        Two kinds of CLIENT state – no server involved. Contrast with{" "}
        <code>/query</code>, which manages SERVER state.
      </p>

      <h2>Theme – React Context</h2>
      <p>
        Current: <strong>{theme}</strong>
      </p>
      <button onClick={toggle}>Toggle theme</button>

      <h2>Counter – Zustand</h2>
      {/* CONCEPT: two separate components read the SAME Zustand store with
        selectors. They share state with zero prop passing and no Provider. */}
      <CounterDisplay />
      <CounterButtons />
    </section>
  );
}

// Reads only `count` via a selector – re-renders only when count changes.
function CounterDisplay() {
  const count = useCounterStore((s) => s.count);
  return (
```

```

    <p>
      Count: <strong>{count}</strong>
    </p>
  );
}

// Reads only the actions. PROD: selecting just the functions means this button
// component never re-renders on count changes - the selective-subscription win.
function CounterButtons() {
  const increment = useCounterStore((s) => s.increment);
  const reset = useCounterStore((s) => s.reset);
  return (
    <div style={{ display: "flex", gap: "0.5rem" }}>
      <button onClick={increment}>+1</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

```

client/src/routes/dashboard/ThemeContext.tsx

```

import { createContext, useContext, useState } from "react";

type Theme = "light" | "dark";
type ThemeContextValue = { theme: Theme; toggle: () => void };

// CONCEPT: Context lets you share a value with any descendant WITHOUT passing it
// through every intermediate component as props ("prop drilling"). Three steps:
// (1) createContext, (2) a Provider supplying the value, (3) useContext to read.
// PROD: the canonical use cases are app-wide, rarely-changing values - the
// current user/auth, theme, locale, feature flags. React's BUILT-IN tool, zero deps.
const ThemeContext = createContext<ThemeContextValue | null>(null);

// (2) The Provider. Anything rendered inside <ThemeProvider> can read the value.
export function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [theme, setTheme] = useState<Theme>("light");
  const toggle = () => setTheme((t) => (t === "light" ? "dark" : "light"));
  // GOTCHA: when this value changes, EVERY component calling useContext here
  // re-renders - Context has no built-in selective subscription. Fine for a
  // theme; a footgun for high-frequency state (that's a reason Zustand exists).
  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

// (3) A typed hook so consumers can't forget the Provider (null check in one place).
export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error("useTheme must be used within <ThemeProvider>");
  return ctx;
}

```

client/src/routes/dashboard/counterStore.ts

```
import { create } from "zustand";

type CounterState = {
  count: number;
  increment: () => void;
  reset: () => void;
};

// CONCEPT: Zustand creates a store OUTSIDE the React tree. No Provider to wrap
// the app, no Context boilerplate. `set` updates state; any component that reads
// the store re-renders when ITS slice changes.
// PROD: the pragmatic default for client state in 2026 - global UI state (modals,
// sidebar open, wizard step, cart) that many unrelated components touch. Lives in
// its own file like this; import the hook anywhere. State + actions colocated.
export const useCounterStore = create<CounterState>((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })),
  reset: () => set({ count: 0 }),
}));

// GOTCHA / the key advantage over Context: components subscribe with a SELECTOR,
// e.g. useCounterStore((s) => s.count). A component selecting only `count` does
// NOT re-render when an unrelated field changes. Context can't do this - every
// consumer re-renders on any change. That selective subscription is why Zustand
// scales to frequently-updated state where Context would thrash.
```

Lesson: mutations + cache invalidation

Route: `/mutate` · **Concept:** writing server state with TanStack Query **Backend:** `POST /api/items`

What it teaches

`useQuery` reads server state; `useMutation` writes it. The pairing - mutate, then **invalidate** the affected query - is how a TanStack app keeps the UI consistent after a write, without hand-syncing state across screens.

This is the production counterpart to `/new-item`. That route used React 19 hooks with a *local* optimistic list (self-contained, no cache). This one writes through the shared cache, so other screens stay in sync.

The flow

1. `useMutation({ mutationFn: createItem })` - the write, triggered by `mutate()`.
2. On success, `queryClient.invalidateQueries({ queryKey: ["items"] })` marks the cached list stale and refetches it.
3. Every component using `["items"]` - here AND on `/query` - re-renders with fresh data.

The production parallel

- **Reads vs writes are different primitives.** `useQuery` runs on render and caches; `useMutation` runs on demand and has side effects. Don't fetch-to-write or write-on-render.
- **The queryKey is the contract.** Because `/query`, `/mutate`, and any future screen all key on `["items"]`, one invalidation refreshes them all. This is why teams standardize query keys (often in a central `queryKeys` factory).
- **Invalidate, don't manually patch.** Rather than "after save, also push the new item into this other component's state," you invalidate and let the cache refetch the truth. Fewer bugs, always consistent with the server.
- **Optimistic mutations** (next level): `onMutate` to update the cache immediately + `onError` to roll back is the cache-integrated version of `/new-item`'s `useOptimistic`.

Try this

1. Add an item - the list updates after the POST + refetch.
2. Open `/query` in a second tab, add an item here, switch to `/query` within 30s - the new item is already there (shared cache, invalidation triggered a refetch).
3. Add a name containing `fail` - the server returns 422, `isError` flips, the list is untouched (no invalidation on failure).

When to use which

- Writing data that other screens read -> **useMutation + invalidateQueries** (this route).
- A self-contained form where the result stays local -> React 19 form hooks (`/new-item`).
- Both can use optimistic UI; `useMutation`'s version also updates the shared cache.

Source

client/src/routes/mutate/Mutate.tsx

```
import { useState } from "react";
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";

type Item = { id: number; name: string };

async function fetchItems(): Promise<Item[]> {
  const res = await fetch("/api/items");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

async function createItem(name: string): Promise<Item> {
  const res = await fetch("/api/items", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name }),
  });
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error ?? `HTTP ${res.status}`);
  }
  return res.json();
}

export default function Mutate() {
  const [name, setName] = useState("");

  // CONCEPT: the QueryClient (the cache) is reachable from any component via this
  // hook. We need it to invalidate cached queries after a successful write.
  const queryClient = useQueryClient();

  // This is the SAME queryKey the /query route uses. Sharing the key means both
  // routes read one cache entry - invalidating it here updates /query too.
  const itemsQuery = useQuery({ queryKey: ["items"], queryFn: fetchItems });

  // CONCEPT: useMutation is the write-side counterpart to useQuery. useQuery is
  // for reads (GET); useMutation is for writes (POST/PUT/DELETE) - things with
  // side effects you trigger explicitly via mutate(), not on render.
  // PROD: this is how server state actually gets updated in a TanStack app. The
  // pattern is: mutate -> on success, invalidate the affected queries so every
  // screen showing that data refetches and stays consistent. No manual state sync.
  const createMutation = useMutation({
    mutationFn: createItem,
    onSuccess: () => {
      // CONCEPT: invalidateQueries marks the cached ["items"] as stale and
      // refetches it. Any component using that key (here AND on /query) updates.
      // PROD: this is the cache-consistency mechanism. One write, every reader
      // refreshes - the reason you don't hand-wire "after save, also update X".
      queryClient.invalidateQueries({ queryKey: ["items"] });
      setName("");
    },
  });

  // GOTCHA: like fetch, the mutationFn must THROW on failure for onError to
```

```

    // fire and isError to flip. A non-throwing 4xx would look like success.
  });

  return (
    <section>
      <h1>Mutate (useMutation + invalidation)</h1>
      <p>
        Add an item with <code>useMutation</code>, then invalidate the{" "}
        <code>["items"]</code> cache. Open <code>/query</code> in another tab and
        add here - that list updates because both share the cache key.
      </p>

      <form
        onSubmit={(e) => {
          e.preventDefault();
          if (name.trim()) createMutation.mutate(name.trim());
        }}
      >
        <input
          value={name}
          onChange={(e) => setName(e.target.value)}
          placeholder="Item name"
          autoComplete="off"
        />
        <button type="submit" disabled={createMutation.isPending}>
          {createMutation.isPending ? "Saving..." : "Add"}
        </button>
      </form>

      {createMutation.isError && (
        <p className="error">Failed: {String(createMutation.error)}</p>
      )}

      {itemsQuery.isPending ? (
        <p>Loading...</p>
      ) : itemsQuery.isError ? (
        <p className="error">Failed to load list</p>
      ) : (
        <ul>
          {itemsQuery.data.map((item) => (
            <li key={item.id}>{item.name}</li>
          ))}
        </ul>
      )}
    </section>
  );
}

```

Lesson: composition + useReducer

Route: /patterns · **Concept:** component composition and complex local state

Part 1 - Composition via `children`

Instead of configuring a component with many props, let the caller pass JSX as `children`. The component owns structure; the caller owns content.

- A `<Card>` that takes `children` works for *any* content - no `title` / `body` props.
- **Compound components** (`Card.Header`) share context by nesting position, not props.
- The type for “anything renderable” is `React.ReactNode`.

Production parallel: this is how design systems are built. Composition over configuration means fewer props, less prop-drilling, and components that flex to cases you didn’t anticipate. Your backend instinct for good interfaces transfers directly - a component’s `children` API is just its public contract.

Part 2 - `useReducer` for complex local state

`useReducer` is `useState`’s bigger sibling: when state has several related fields and multiple transitions, centralize the logic in one pure reducer function.

- `dispatch({ type: "increment" })` describes *what happened*; the reducer decides *how state changes*.
- It’s the same reducer pattern as Redux, built into React, scoped to one component.

Production parallel: reach for it when `useState` calls multiply and start depending on each other (forms with interdependent fields, wizards, toggles with rules). One pure function is easier to test and reason about than scattered `setState`.

When to use which

- One or two independent values -> `useState`.
- Several related fields with structured transitions -> `useReducer`.
- Shared across many components -> Zustand / Context (`/dashboard`).
- Server data -> TanStack Query (`/query`, `/mutate`).

Note: memoization is automatic now (React Compiler)

You will NOT find `useMemo`, `useCallback`, or `React.memo` anywhere in this project. That is intentional. The **React Compiler** (stable, v1.0, Oct 2025) auto-memoizes at build time - it analyzes which values depend on which state and inserts memoization for you.

Before (pre-2025): you hand-wrapped expensive calculations and callbacks:

```
const sorted = useMemo(() => items.sort(byName), [items]);
const onClick = useCallback(() => doThing(id), [id]);
```

Now: write the plain version; the compiler handles it.

```
const sorted = items.sort(byName); // compiler memoizes if needed
const onClick = () => doThing(id);
```

Existing `useMemo` / `useCallback` in old codebases are safe but redundant. Knowing this - and deliberately NOT reaching for manual memoization - is itself the senior move.

Source

client/src/routes/patterns/Patterns.tsx

```
import { useReducer } from "react";

// -----
// PART 1: COMPOSITION via children
// -----

// CONCEPT: instead of configuring a component with a dozen props, you let the
// CALLER pass JSX as `children`. The component owns the structure/styling; the
// caller owns the content. "Composition over configuration."
// PROD: this is how design systems are built - a <Card> doesn't take a `title`
// and `body` prop, it takes children, so it works for any content. Fewer props,
// infinitely flexible. React.ReactNode is the type for "any renderable thing."
function Card({ children }: { children: React.ReactNode }) {
  return (
    <div
      style={{
        border: "1px solid #e2e2e2",
        borderRadius: 6,
        padding: "1rem",
        marginBottom: "1rem",
      }}
    >
      {children}
    </div>
  );
}

// A compound component: Card.Header and Card.Body share the Card's context of
// "being inside a card" purely by where the caller places them. No prop wiring.
Card.Header = function CardHeader({ children }: { children: React.ReactNode }) {
  return <div style={{ fontWeight: 600, marginBottom: "0.5rem" }}>{children}</div>;
};

// -----
// PART 2: useReducer for complex local state
// -----

type State = { count: number; step: number };
type Action =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "setStep"; step: number }
  | { type: "reset" };

// CONCEPT: useReducer is useState's bigger sibling. When state has multiple
// related fields and several ways to transition, a reducer centralizes that
// logic in one pure function instead of scattering setState calls.
// PROD: if you know Redux, this IS Redux's reducer pattern, built into React and
// scoped to one component. Reach for it when useState branches get tangled, or
// when the next transition depends on the current state in non-trivial ways.
function reducer(state: State, action: Action): State {
  switch (action.type) {
```

```

    case "increment":
      return { ...state, count: state.count + state.step };
    case "decrement":
      return { ...state, count: state.count - state.step };
    case "setStep":
      return { ...state, step: action.step };
    case "reset":
      return { count: 0, step: 1 };
  }
}

export default function Patterns() {
  // dispatch sends an action; the reducer computes the next state. Components
  // describe WHAT happened ("increment"), not HOW state changes - that's in the reducer.
  const [state, dispatch] = useReducer(reducer, { count: 0, step: 1 });

  return (
    <section>
      <h1>Patterns (composition + useReducer)</h1>

      <Card>
        <Card.Header>Composition</Card.Header>
        <p>
          This card takes no <code>title</code>/<code>body</code> props - it
          renders whatever children you nest inside it.
        </p>
      </Card>

      <Card>
        <Card.Header>useReducer</Card.Header>
        <p>
          Count: <strong>{state.count}</strong> (step {state.step})
        </p>
        <div style={{ display: "flex", gap: "0.5rem", marginBottom: "0.5rem" }}>
          <button onClick={() => dispatch({ type: "decrement" })}>-</button>
          <button onClick={() => dispatch({ type: "increment" })}>+</button>
          <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
        </div>
        <label>
          Step:{" "}
          <input
            type="number"
            value={state.step}
            onChange={(e) =>
              dispatch({ type: "setStep", step: Number(e.target.value) || 1 })
            }
            style={{ width: 60 }}
          />
        </label>
      </Card>
    </section>
  );
}

```

client/src/routes/patterns/Patterns.test.tsx

```
import { fireEvent, render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import Patterns from "../Patterns";

// CONCEPT: React Testing Library tests BEHAVIOR through the rendered UI, the way
// a user experiences it – find elements by their visible text/role, click them,
// assert on what changed. It deliberately does NOT let you reach into component
// internals (state, the reducer fn). If a test passes, the user-facing behavior works.
// PROD: this is the standard React testing approach. Tests survive refactors
// because they assert on outcomes ("count shows 2"), not implementation
// ("useReducer was called"). Rewrite the component internals and the test still holds.

describe("Patterns – useReducer behavior", () => {
  it("increments the count by the current step", async () => {
    const user = userEvent.setup();
    render(<Patterns />);

    // Find by visible text, like a user would. Starts at 0.
    expect(screen.getByText(/Count:/)).toHaveTextContent("Count: 0");

    // getByRole("button", { name }) finds the button by its accessible label.
    await user.click(screen.getByRole("button", { name: "+" }));
    expect(screen.getByText(/Count:/)).toHaveTextContent("Count: 1");
  });

  it("respects a changed step value", async () => {
    const user = userEvent.setup();
    render(<Patterns />);

    const stepInput = screen.getByLabelText(/Step:/);
    // fireEvent.change sets the value atomically – the reliable way to drive a
    // controlled input when simulating per-keystroke typing is flaky (here a
    // number input in jsdom). It fires a single onChange with the final value.
    fireEvent.change(stepInput, { target: { value: "5" } });

    await user.click(screen.getByRole("button", { name: "+" }));
    expect(screen.getByText(/Count:/)).toHaveTextContent("Count: 5");
  });

  it("resets count and step", async () => {
    const user = userEvent.setup();
    render(<Patterns />);

    await user.click(screen.getByRole("button", { name: "+" }));
    await user.click(screen.getByRole("button", { name: "Reset" }));
    expect(screen.getByText(/Count:/)).toHaveTextContent("Count: 0");
  });
});
```

Lesson: session management

Route: `/auth` · **Concept:** browser sessions, JWT access + refresh, httpOnly cookies **Backend:** `POST /api/login`, `POST /api/refresh`, `POST /api/logout`, `GET /api/me`

In production you would **NOT** write this by hand. Use a library. This lesson builds a minimal version so the libraries stop being magic - so you can configure them correctly and debug them when sessions misbehave.

The flow this implements

1. **Login** - server verifies credentials, sets TWO httpOnly cookies: a short-lived **access token** (JWT-like, 10s here) and a long-lived, server-tracked **refresh token**.
2. **Authenticated requests** - the browser auto-sends the cookies. JS never reads them (that's what `httpOnly` means).
3. **Access token expires** - `GET /api/me` returns 401. `authFetch` calls `/api/refresh` once, gets a fresh access token, and retries the request.
4. **Logout** - clears the cookies AND deletes the refresh token server-side (revocation).

The two axes (don't conflate them)

- **Token format:** opaque session id (stateful, instantly revocable) vs JWT (stateless, fast, but can't be revoked before expiry).
- **Browser storage:** httpOnly cookie (XSS-safe, needs CSRF defense) vs localStorage (readable by any JS = XSS steals it - avoid) vs in-memory.

The production sweet spot, and what this lesson does: **short-lived JWT access token + long-lived, server-tracked refresh token, both in httpOnly cookies**. Stateless speed per request, plus revocation via the refresh store.

The pieces (and the production parallel)

- **authFetch** - the 401 -> refresh -> retry interceptor, with a **single-flight guard** so concurrent 401s share ONE refresh (the bug most hand-rolled versions have). PROD: this is what an axios interceptor / auth library middleware does for you.
- **AuthContext** - app-wide current user via `useAuth()`. PROD: a library's `<AuthProvider>` + `useUser()` / `useSession()`.
- **Route guard** - render login vs protected based on session. PROD: often a React Router `loader` that redirects before render (see the `/items` lesson).

- **Refresh-token store** (server `Set`) - the stateful half that enables revocation. PROD: Redis or a DB table.

Libraries (2026) - what you'd actually use

Tool	Type	Use when
Better Auth	self-hosted lib	own your data, TS-native, modern stack
Clerk	hosted service	want it done, pre-built React UI, ship fast
Supabase Auth	bundled backend	already on Supabase (DB-enforced authz)
Auth0	hosted service	enterprise SSO/SAML, compliance
Auth.js	self-hosted lib	established incumbent; less the default for new apps

They all implement the flow above internally (httpOnly cookies, refresh rotation, CSRF). The senior move is knowing the pattern well enough to pick and configure the right one - not to ship your own.

In an actual project, reach for [Better Auth \(docs\)](#) - TypeScript-native, you own the data, and it handles the access/refresh/httpOnly-cookie flow this lesson builds by hand. The hand-rolled version here is only to show you what it does internally.

Try this

1. Log in (creds pre-filled) - the protected view appears.
2. Click **Call /api/me** immediately - `200 OK`.
3. Wait ~10s (access token expires), click **Call /api/me** again - it still returns `200`, because the first call 401'd, `authFetch` refreshed, and retried. Watch the Network tab: you'll see `me (401)` -> `refresh` -> `me (200)`.
4. Log out, then **Call /api/me** - `401` (refresh token revoked server-side).

When to use which token strategy

- One app, one backend, want easy revocation -> **opaque server session** in an httpOnly cookie. The simplest correct default.
- SPA + separate API, or services that verify locally -> **JWT access + refresh** (this lesson).
- Non-browser client (mobile, m2m) -> JWT in an `Authorization` header, no cookie.
- Any browser app -> **don't put tokens in localStorage**.

Source

client/src/routes/auth/Auth.tsx

```
import { useState } from "react";
import { AuthProvider, useAuth } from "../AuthContext";

// This route wraps its own subtree in AuthProvider so the lesson is
// self-contained. PROD: you'd hoist <AuthProvider> to the app root (main.tsx)
// so the whole app shares one session, exactly like a library's provider.
export default function Auth() {
  return (
    <AuthProvider>
      <AuthInner />
    </AuthProvider>
  );
}

function AuthInner() {
  const { user, loading } = useAuth();

  // CONCEPT: a route guard. While we check the session, render nothing
  // definitive; once known, show either the protected view or the login form.
  // PROD: with React Router this guard is often a route `loader` that redirects
  // to /login before the component renders (ties back to the /items lesson).
  if (loading) return <Shell><p>Checking session...</p></Shell>;
  return user ? <Protected /> : <LoginForm />;
}

function LoginForm() {
  const { login } = useAuth();
  const [email, setEmail] = useState("demo@example.com");
  const [password, setPassword] = useState("password");
  const [error, setError] = useState<string | null>(null);

  async function onSubmit(e: React.FormEvent) {
    e.preventDefault();
    setError(null);
    try {
      await login(email, password);
    } catch {
      setError("Invalid credentials");
    }
  }

  return (
    <Shell>
      <p>Log in (demo creds are pre-filled).</p>
      <form onSubmit={onSubmit} style={{ display: "grid", gap: "0.5rem", maxWidth: 280 }}>
        <input value={email} onChange={(e) => setEmail(e.target.value)} placeholder="Email" />
        { /* PROD: a password field - the browser handles it; we never store it. */ }
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="Password"
        />
      </form>
    </Shell>
  );
}
```



```

    />
    <button type="submit">Log in</button>
  </form>
  {error && <p className="error">{error}</p>}
</Shell>
);
}

function Protected() {
  const { user, logout } = useAuth();
  return (
    <Shell>
      <p>
        You are logged in as <strong>{user?.email}</strong>. This view is gated by
        the session.
      </p>
      <p>
        The access token expires after 10s. Wait, then click "Call /api/me" – the
        first call 401s, authFetch silently refreshes, and the retry succeeds.
      </p>
      <CallMe />
      <button onClick={() => logout()}>Log out</button>
    </Shell>
  );
}

function CallMe() {
  const [result, setResult] = useState<string>("");
  async function call() {
    // Use authFetch so the 401 -> refresh -> retry happens transparently.
    const { authFetch } = await import("./authFetch");
    const res = await authFetch("/api/me");
    setResult(res.ok ? `200 OK: ${JSON.stringify(await res.json())}` : `${res.status}`);
  }
  return (
    <div style={{ margin: "0.5rem 0" }}>
      <button onClick={call}>Call /api/me</button>
      {result && <p><code>{result}</code></p>}
    </div>
  );
}

function Shell({ children }: { children: React.ReactNode }) {
  return (
    <section>
      <h1>Auth (session management)</h1>
      {children}
    </section>
  );
}

```

client/src/routes/auth/AuthContext.tsx

```
import { createContext, useContext, useEffect, useState } from "react";
import { authFetch } from "../authFetch";

type User = { id: number; email: string };
type AuthContextValue = {
  user: User | null;
  loading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
};

// CONCEPT: the auth context - the /dashboard Context pattern applied to a real
// concern. Any component can call useAuth() to read the current user or log in/out.
// PROD: this is what a library's <AuthProvider> + useUser()/useSession() give you.
const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  // On mount, ask the server who we are. authFetch will silently refresh if the
  // access token is stale. This is how the session "survives a page reload"
  // even though JS never sees the httpOnly token.
  useEffect(() => {
    authFetch("/api/me")
      .then((res) => (res.ok ? res.json() : null))
      .then((data) => setUser(data?.user ?? null))
      .catch(() => setUser(null))
      .finally(() => setLoading(false));
  }, []);

  async function login(email: string, password: string) {
    const res = await authFetch("/api/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password }),
    });
    if (!res.ok) throw new Error("invalid credentials");
    const data = await res.json();
    setUser(data.user);
  }

  async function logout() {
    await authFetch("/api/logout", { method: "POST" });
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, loading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
```

```

const ctx = useContext(AuthContext);
if (!ctx) throw new Error("useAuth must be used within <AuthProvider>");
return ctx;
}

```

client/src/routes/auth/authFetch.ts

```

// CONCEPT: a fetch wrapper that transparently handles access-token expiry.
// On a 401 it calls /api/refresh once, then retries the original request.
// PROD: this is exactly the "interceptor" an auth library installs for you
// (axios interceptor / library middleware). You rarely write it by hand - but
// understanding it is how you debug "why am I randomly getting logged out".

// GOTCHA: the single-flight guard. When the access token expires, SEVERAL
// requests can 401 at once. Without this, each fires its own /refresh, racing
// and rotating tokens chaotically. We keep ONE in-flight refresh promise and
// make every concurrent 401 await the same one.
let refreshPromise: Promise<boolean> | null = null;

async function refreshOnce(): Promise<boolean> {
  if (!refreshPromise) {
    refreshPromise = fetch("/api/refresh", {
      method: "POST",
      credentials: "include", // send the refresh cookie
    })
      .then((res) => res.ok)
      .finally(() => {
        refreshPromise = null; // reset so the next expiry can refresh again
      });
  }
  return refreshPromise;
}

export async function authFetch(
  input: RequestInfo,
  init: RequestInit = {},
): Promise<Response> {
  // credentials: "include" makes the browser send our httpOnly auth cookies.
  // (We never read the token in JS - that's the point of httpOnly.)
  const opts: RequestInit = { ...init, credentials: "include" };

  let res = await fetch(input, opts);
  if (res.status !== 401) return res;

  // Access token likely expired - try one refresh, then replay the request.
  const refreshed = await refreshOnce();
  if (!refreshed) return res; // refresh failed -> caller treats as logged out
  res = await fetch(input, opts);
  return res;
}

```